

Jenkins-Task-3

1. Create one Jenkins job using the below code and create three stages.

- **stage1:** Git clone to download the source code.
- **stage2:** Sonarqube Integration to check the quality of code.
- **stage3:** Slack Integration to send the alerts to slack.

Source Code: <https://github.com/betawins/VProfile-1.git>

SonarQube Server Installation & Configuration Guide

Target OS: Amazon Linux (AL2 / AL2023)

Database: PostgreSQL 14/15

Runtime Environment: Java 21 (Amazon Corretto)

`sudo dnf install java-21-amazon-corretto -y`

Application Account Context: Default ec2-user

Ports:

PostgreSQL 5432

SonarQube 9000

Objective

The primary objective of this project is to build an automated Continuous Integration (CI) pipeline that standardizes code quality audits and real-time team notifications.

Purpose

The purpose of this project is to create an automated workflow that checks code for bugs and updates the development team instantly.

Stage-1

Created a job

New Item → Name → Freestyle → Source Code Management → Git → installed git in Ec2 Jenkins master server also → Save → Build Now

Stage-2

1.Install PostgreSQL Packages

`sudo dnf update -y`

`sudo dnf install postgresql15 postgresql15-server -y`

2.Initialize & Start the Database

Initialize the database layout

`sudo postgresql-setup --initdb`

Start and enable PostgreSQL to launch on boot

`sudo systemctl start postgresql`

`sudo systemctl enable postgresql`

3. Login to PostgreSQL Administrator

Login : `sudo -i -u postgres psql`

This opens the PostgreSQL shell: `postgres=#`

Jenkins-Task-3

4. Create SonarQube Database

Create the database: `CREATE DATABASE sonarqube;`

Check databases: `\l`

The database should appear as: `sonarqube`

3. Create SonarQube PostgreSQL User

-- Create a dedicated system user account for SonarQube

`CREATE USER sonar WITH PASSWORD 'YourStrongPassword';`

Create the SonarQube database user:

`CREATE USER sonar WITH PASSWORD 'Sonar@123';`

Grant the user access to the database:

`GRANT ALL PRIVILEGES ON DATABASE sonarqube TO sonar;`

Check users:

`\du`

The sonar user should appear.

4. PostgreSQL Authentication Configuration

PostgreSQL uses `pg_hba.conf` to control how users authenticate.

Check the location:

`sudo -l -u postgres psql -c "SHOW hba_file;"`

Current configuration file:

`/var/lib/pgsql/data/pg_hba.conf`

Edit the file:

`sudo vi /var/lib/pgsql/data/pg_hba.conf`

For local TCP connections, configure:

# TYPE	DATABASE	USER	ADDRESS	METHOD
# Local Unix socket				
local	all	all		peer
# IPv4 local connections				
host	all	all	127.0.0.1/32	scram-sha-256
# IPv6 local connections				
host	all	all	:::1/128	scram-sha-256

The important part is: `scram-sha-256`

PostgreSQL uses one database role (`sonar`) and controls connection access through `pg_hba.conf`.

5. Restart PostgreSQL

After changing `pg_hba.conf`:

`sudo systemctl restart postgresql`

Check the service:

`sudo systemctl status postgresql`

Jenkins-Task-3

6. Set SonarQube Database Password

Login as PostgreSQL administrator:

```
sudo -i -u postgres psql
```

Set the password:

```
ALTER USER sonar WITH PASSWORD 'Sonar@123';
```

Expected: ALTER ROLE

Exit: \q

PostgreSQL does not require FLUSH PRIVILEGES;. That is a MySQL/MariaDB command. PostgreSQL applies role and privilege changes immediately.

7. Test SonarQube Database Login

Test the database using the sonar user:

```
psql -U sonar -h 127.0.0.1 -d sonarqube
```

Enter the password when prompted:

Password for user sonar: Password: Sonar@123

Successful login should show: sonarqube=>

Verify the current user: SELECT current_user;

Expected: sonar

Exit: \q

8. SonarQube Version

Official download URL used:

<https://binaries.sonarsource.com/Distribution/sonarqube/sonarqube-26.9.0.129388.zip>

→ sudo unzip sonarqube-26.9.0.129388.zip

→ sudo mv sonarqube-26.9.0.129388 sonar

Correct File Ownership & File Permissions

```
sudo chown -R ec2-user:ec2-user /opt/sonar
```

```
sudo chmod +x /opt/sonar/bin/linux-x86-64/sonar.sh
```

{if incase} Clear out any residual corrupted background lock spaces:

```
sudo rm -rf /opt/sonar/temp/*
```

Section 1: Host-Level Prerequisites & Kernel Tweaks

```
sudo vi /etc/sysctl.conf
```

```
vm.max_map_count=524288
```

```
fs.file-max=131072 (Save and close the file )
```

Force the Linux kernel to apply the virtual memory rules immediately without a system reboot:

```
sudo sysctl -p
```

Jenkins-Task-3

Section 2: Database Layer Configuration (PostgreSQL)

Step 2.2: Build the SonarQube Database Schema

Log into the local database: `sudo -i -u postgres psql`

-- Create the dedicated application database space

```
CREATE DATABASE sonarqube WITH ENCODING 'UTF8';
```

-- Provision the structural software credentials

```
CREATE USER sonar WITH PASSWORD 'Sonar@123';
```

-- Assign global database privileges to the new identity

```
GRANT ALL PRIVILEGES ON DATABASE sonarqube TO sonar;
```

Confirm the configuration and exit the engine:

```
→ \l
```

```
→ \du
```

```
→ \q
```

Section 3: App Server Deployment & Core Settings

Step 3.: Map Database Connectivity Settings

`vi /opt/sonar/conf/sonar.properties`

```
sonar.jdbc.username=sonar
```

```
sonar.jdbc.password=Sonar@123
```

```
sonar.jdbc.url=jdbc:postgresql://localhost:5432/sonarqube
```

```
sonar.web.host=0.0.0.0
```

```
sonar.web.context=/sonar
```

Save and close

Section 4: Daemon Automation & Service Operations

Step 4.1: Establish the background systemd Unit Script

Create a system-managed orchestration daemon wrapper file:

`sudo vi /etc/systemd/system/sonarqube.service`

```
[Unit]
```

```
Description=SonarQube service
```

```
After=syslog.target network.target postgresql.service
```

```
[Service]
```

```
Type=forking
```

```
ExecStart=/opt/sonar/bin/linux-x86-64/sonar.sh start
```

```
ExecStop=/opt/sonar/bin/linux-x86-64/sonar.sh stop
```

```
User=ec2-user
```

```
Group=ec2-user
```

```
Restart=always
```

```
LimitNOFILE=65536
```

```
LimitNPROC=4096
```

```
[Install]
```

```
WantedBy=multi-user.target
```

Jenkins-Task-3

Step 4.2: Execute Process Orchestration

Notify the operating system engine of changes, reset tracking failures, and spin up the live process:

→ `sudo systemctl daemon-reload`

→ `sudo systemctl reset-failed sonarqube.service`

→ `sudo systemctl restart sonarqube`

→ `sudo systemctl enable sonarqube`

Confirm the server status is stable, green, and reporting **active (running)**:

→ `sudo systemctl status sonarqube`

Section 5: Post-Deployment Verification & UI Operations

Step 5.1: Confirm Port Network Binding

Run a network socket scan to ensure the application server has successfully occupied network port **9000**:

`sudo ss -tulnp | grep 9000`

Step 5.2: Launch the Web UI Dashboard Interface

- **URL Address:** `http://<YOUR_EC2_PUBLIC_IP>:9000/sonar`
- **Default Username:** `admin`
- **Default Password:** `admin`

Stage-2

Section 6: Slack Integration Guide

Step 6.1: Create a Slack Channel

1. Go to slack.com and make a free workspace named **DevOps-Test**.
2. Open the workspace. In the left sidebar, click **Add channels** ➡ **Create a new channel**.
3. Name the channel **jenkins-builds**.
4. Make sure it is set to **Public**, then click **Create**.

Step 6.2: Create the Slack App Bot

1. Open a new browser tab and go to slack.com.
2. Click **Create New App**, and select **From scratch**.
3. Name the app **Jenkins Bot** and select your **DevOps-Test** workspace. Click **Create App**.
4. In the left menu, click **OAuth & Permissions**.
5. Scroll down to the **Bot Token Scopes** section, click **Add an OAuth Scope**, and add these two permissions:
 - **chat:write**
 - **chat:write.public**
6. Scroll back to the top of the page and click the green **Install to Workspace** button.
7. Click **Allow** on the permission screen.
8. Copy the **Bot User OAuth Token** that appears (it starts with **xoxb-**).

Step 6.3: Add the Token to Jenkins

1. Open your **Jenkins Dashboard** tab.
2. Go to **Manage Jenkins** ➡ **Credentials** ➡ **(global)** ➡ **Add Credentials** (top right).
3. Fill out the fields exactly like this:
 - **Kind:** Secret text
 - **Secret:** Paste your **xoxb-** token.
 - **ID:** slack-token
 - **Description:** Slack Bot Token

Jenkins-Task-3

4. Click **Create**.

Step 6.4: Configure Jenkins System Settings

1. Go to **Manage Jenkins** → **System**.
2. Scroll to the bottom to find the **Slack** section.
3. Fill out the configuration exactly like this:
 - **Workspace:** devops-test
 - **Credential:** Select **slack-token** from the dropdown menu.
 - **Default channel:** #jenkins-builds
 - **Custom slack app bot user:** **CHECK THIS BOX.**
4. Click **Test Connection**. You will instantly see a **"Success"** message appear in your Slack channel.
5. Scroll to the bottom of the Jenkins page and click **Save**.

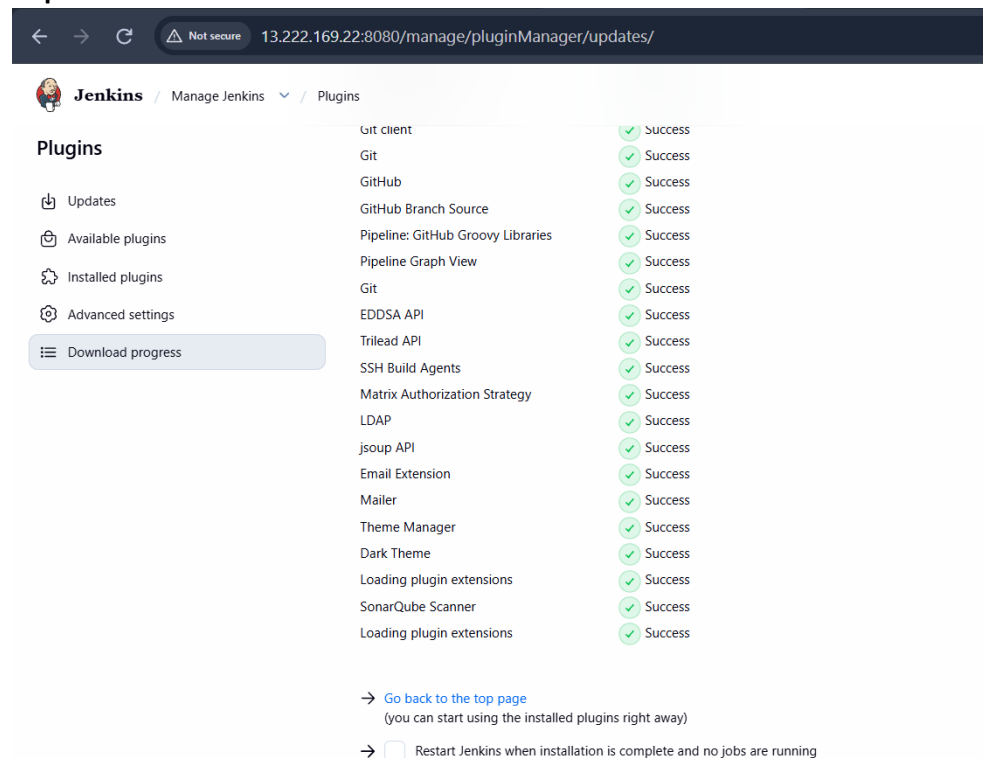
Step 6.5: Enable Slack Alerts in your Job

1. Open your project job (**stage_1**) and click **Configure**.
2. Scroll to the absolute bottom to the **Post-build Actions** section.
3. Click **Add post-build action** and choose **Slack Notifications**.
4. Check the boxes for **Notify Success** and **Notify Failure**.
5. Click **Save**.

Step 6.6: Run a Test Build

Now we have [The Output in Workspace Channel](#)

Implementation:



← → ↻ ⚠ Not secure 13.222.169.22:8080/manage/pluginManager/updates/

Jenkins / Manage Jenkins ▾ / Plugins

Plugins	Status
Git client	✓ Success
Git	✓ Success
GitHub	✓ Success
GitHub Branch Source	✓ Success
Pipeline: GitHub Groovy Libraries	✓ Success
Pipeline Graph View	✓ Success
Git	✓ Success
EDDSA API	✓ Success
Trilead API	✓ Success
SSH Build Agents	✓ Success
Matrix Authorization Strategy	✓ Success
LDAP	✓ Success
jsoup API	✓ Success
Email Extension	✓ Success
Mailer	✓ Success
Theme Manager	✓ Success
Dark Theme	✓ Success
Loading plugin extensions	✓ Success
SonarQube Scanner	✓ Success
Loading plugin extensions	✓ Success

→ [Go back to the top page](#)
(you can start using the installed plugins right away)

→ ☐ Restart Jenkins when installation is complete and no jobs are running

Jenkins-Task-3

```
/root
Last login: Thu Oct 1 10:37:28 2026 from 18.206.107.29
[ec2-user@ip-172-31-16-204 ~]$ sudo dnf update -y
Amazon Linux 2023 Kernel Livepatch repository
Dependencies resolved.
Nothing to do.
Complete!
[ec2-user@ip-172-31-16-204 opt]$ wget https://binaries.sonarsource.com/Distribution/sonarqube/sonarqube-6.6.zip
--2026-10-01 10:50:08-- https://binaries.sonarsource.com/Distribution/sonarqube/sonarqube-6.6.zip
Resolving binaries.sonarsource.com (binaries.sonarsource.com)... 3.167.37.121, 3.167.37.88, 3.167.37.95, ...
Connecting to binaries.sonarsource.com (binaries.sonarsource.com)|3.167.37.121|:443... connected.
HTTP request sent, awaiting response... 403 Forbidden
2026-10-01 10:50:08 ERROR 403: Forbidden.

[ec2-user@ip-172-31-16-204 opt]$ sudo su
[root@ip-172-31-16-204 opt]# wget https://binaries.sonarsource.com/Distribution/sonarqube/sonarqube-6.6.zip
--2026-10-01 10:50:29-- https://binaries.sonarsource.com/Distribution/sonarqube/sonarqube-6.6.zip
Resolving binaries.sonarsource.com (binaries.sonarsource.com)... 3.167.37.10, 3.167.37.95, 3.167.37.88, ...
Connecting to binaries.sonarsource.com (binaries.sonarsource.com)|3.167.37.10|:443... connected.
HTTP request sent, awaiting response... 403 Forbidden
2026-10-01 10:50:30 ERROR 403: Forbidden.

[root@ip-172-31-16-204 opt]# sudo dnf install java-21-amazon-corretto -y
Last metadata expiration check: 0:18:07 ago on Thu Oct 1 10:38:52 2026.
Dependencies resolved.

Package Architecture Version
Installing:
java-21-amazon-corretto x86_64 1:21.0.12+9-1.amzn2023.1
Installing dependencies:
alsa-lib x86_64 1.2.7.2-1.amzn2023.0.4
cairo x86_64 1.18.0-4.amzn2023.0.3
dejavu-sans-fonts noarch 2.37-16.amzn2023.0.2
dejavu-sans-mono-fonts noarch 2.37-16.amzn2023.0.2
dejavu-serif-fonts noarch 2.37-16.amzn2023.0.2
fontconfig x86_64 2.13.94-2.amzn2023.0.2
fonts-filesystem noarch 1:2.0.5-12.amzn2023.0.2
freetype x86_64 2.13.2-5.amzn2023.0.2
```

i-0a04c6a95573d186f (Sonarqube-server)

PublicIPs: 18.212.205.252 PrivateIPs: 172.31.16.204

```
postgres=# \l

          List of databases
  Name      | Owner   | Encoding | Collate | Ctype   | ICU Locale | Locale Provider | Access privileges
-----+-----+-----+-----+-----+-----+-----+-----
postgres   | postgres | UTF8     | C.UTF-8 | C.UTF-8 |             | libc            | 
sonarqube  | postgres | UTF8     | C.UTF-8 | C.UTF-8 |             | libc            | =Tc/postgres+
           |          |          |          |          |             |                  | postgres=Ctc/postgres+
           |          |          |          |          |             |                  | sonar=Ctc/postgres
template0  | postgres | UTF8     | C.UTF-8 | C.UTF-8 |             | libc            | =c/postgres+
           |          |          |          |          |             |                  | postgres=Ctc/postgres
template1  | postgres | UTF8     | C.UTF-8 | C.UTF-8 |             | libc            | =c/postgres+
           |          |          |          |          |             |                  | postgres=Ctc/postgres
(4 rows)
```

```
postgres=# \du

          List of roles
 Role name | Attributes                                  | Member of
-----+-----+-----
postgres  | Superuser, Create role, Create DB, Replication, Bypass RLS | {}
sonar     |                               | {}
```

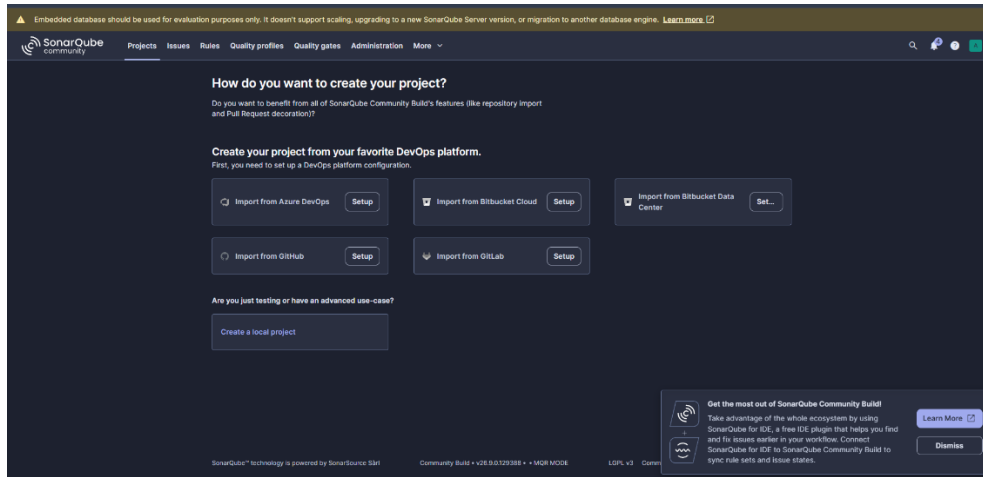
```
postgres=# \q
[root@ip-172-31-16-204 /]#
```

```
# SIGNOFF THE SERVER FOR THE CHANGES TO TAKE EFFECT, RUN "pg_ctl reload",
# or execute "SELECT pg_reload_conf()".
#
# Put your actual configuration here
# -----
#
# If you want to allow non-local connections, you need to add more
# "host" records. In that case you will also need to make PostgreSQL
# listen on a non-local interface via the listen_addresses
# configuration parameter, or via the -i or -h command line switches.


# TYPE      DATABASE      USER      ADDRESS      METHOD

# "local" is for Unix domain socket connections only
local      all             all                                     peer
# IPv4 local connections:
host       all             all             127.0.0.1/32  ident
# IPv6 local connections:
host       all             all             ::1/128      ident
# Allow replication connections from localhost, by a user with the
# replication privilege.
local      replication    all                                     peer
host       replication    all             127.0.0.1/32  scram-sha-256
host       replication    all             ::1/128      scram-sha-256
-- INSERT --
```

Jenkins-Task-3



```
# Format of deprecation log.
# See https://docs.sonarsource.com/sonarqube/latest/instance-administration/server-logs-and-system-info/
# The login of authenticated user is disabled by default.
# sonar.deprecationLogs.loginEnabled=false

#-----
# OTHERS

# Delay in seconds between processing of notification queue. Default is 60 seconds.
#sonar.notifications.delay=60

# Paths to persistent data files (embedded database and search index) and temporary files.
# Can be absolute or relative to installation directory.
# Defaults are respectively <installation home>/data and <installation home>/temp
#sonar.path.data=data
#sonar.path.temp=temp

# Telemetry - Share anonymous SonarQube statistics
# By sharing anonymous SonarQube statistics, you help us understand how SonarQube is used so we can improve the product to work even better for you.
# We don't collect source code or IP addresses. And we don't share the data with anyone else.
#sonar.telemetry.enable=true

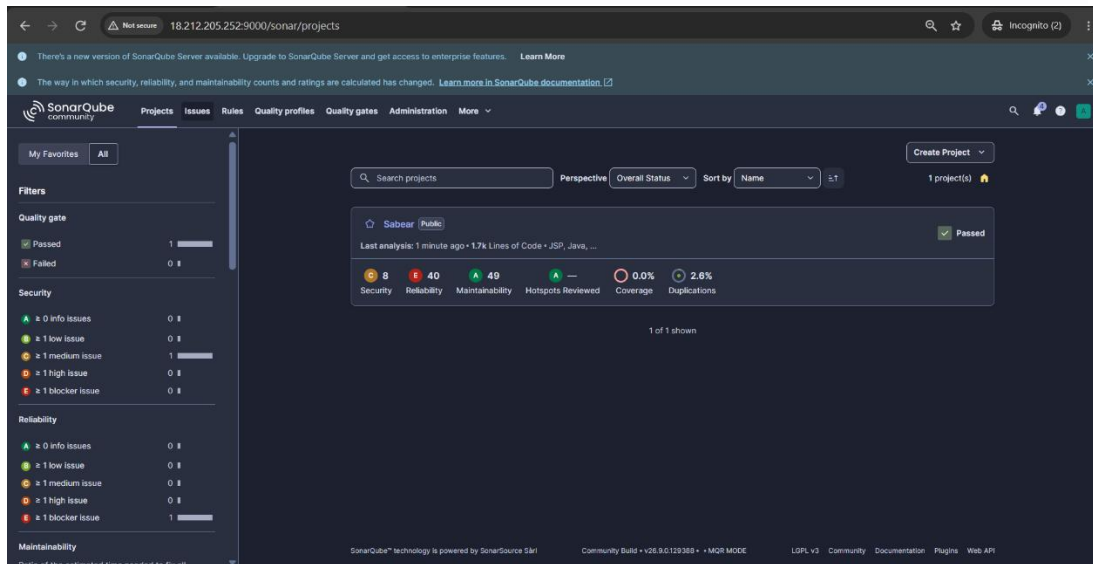
sonar.jdbc.username=sonar
sonar.jdbc.password=Sonar@123
sonar.jdbc.url=jdbc:postgresql://localhost:5432/sonarqube

sonar.web.host=0.0.0.0
sonar.web.port=9000

-- INSERT --
```

i-0a04c6a95573d186f (Sonarqube-server)

PublicIPs: 18.212.205.252 PrivateIPs: 172.31.16.204



Jenkins-Task-3

```
Jenkins / stage_1 / #5  
16:22:18.240 INFO 2/2 source files have been analyzed  
16:22:18.716 INFO Hit the cache for 0 out of 1  
16:22:18.729 INFO Miss the cache for 1 out of 1: ANALYSIS_MODE_UNELECTABLE [1/1]  
16:22:18.754 INFO Sensor JavaScript/TypeScript/CSS analysis [javascript] (done) | time=23219ms  
16:22:18.744 INFO Sensor IAc Project Sensor [iac]  
16:22:18.810 INFO Sensor IAc Project Sensor [iac] (done) | time=64ms  
16:22:18.817 INFO Sensor JavaProjectSensor [java]  
16:22:18.801 INFO Sensor JavaProjectSensor [java] (done) | time=70ms  
16:22:18.891 INFO Sensor Zero Coverage Sensor  
16:22:19.025 INFO Sensor Zero Coverage Sensor (done) | time=142ms  
16:22:19.053 INFO Sensor Java CPD Block Indexer  
16:22:19.670 INFO Sensor Java CPD Block Indexer (done) | time=640ms  
16:22:19.673 INFO ----- Gather SCA dependencies on project  
16:22:20.090 INFO Dependency analysis skipped  
16:22:20.134 INFO SCM Publisher SCM provider for this project is: git  
16:22:20.450 INFO SCM Publisher 31 source files to be analyzed  
16:22:21.854 INFO SCM Publisher 31/31 source files have been analyzed (done) | time=1705ms  
16:22:21.944 INFO CPD Executor 8 files had no CPD blocks  
16:22:21.946 INFO CPD Executor Calculating CPD for 14 files  
16:22:21.966 INFO CPD Executor CPD calculation finished (done) | time=69ms  
16:22:21.996 INFO SCM revision ID 'afcl37367641aaafae82081ae71ac0839081'  
16:22:23.000 INFO Analysis report generated in 881ms, dir size=680.9 kB  
16:22:23.162 INFO Analysis report compressed in 152ms, zip size=150.8 kB  
16:22:24.040 INFO Analysis report uploaded in 800ms  
16:22:24.050 INFO ANALYSIS SUCCESSFUL, you can find the results at: http://18.232.205.252:9000/sonar/dashboard?id=Subear  
16:22:24.059 INFO Note that you will be able to access the updated dashboard once the server has processed the submitted analysis report  
16:22:24.060 INFO More about the report processing at http://18.232.205.252:9000/sonar/api/ci/task?id=6d2fa59c-a66f-46c8-b729-20f78f55700a  
16:22:24.883 INFO Analysis total time: 41.163 s  
16:22:24.915 INFO SonarScanner Engine completed successfully  
16:22:25.035 INFO EXECUTION SUCCESS  
16:22:25.189 INFO Total time: 48.480s  
Finished: SUCCESS
```

Jenkins / stage_1

Status stage_1

Changes

Workspace

Build Now

Delete Project

Configure

Rename

SonarQube

SonarQube Quality Gate

Sabear Passed

server-side processing: Success

Permalinks

- Last build (#5), 3 min 28 sec ago
- Last stable build (#5), 3 min 28 sec ago
- Last successful build (#5), 3 min 28 sec ago
- Last failed build (#1), 2 hr 51 min ago
- Last unsuccessful build (#1), 2 hr 51 min ago
- Last completed build (#5), 3 min 28 sec ago

Builds >

Filter

Today

- #5 16:21
- #1 13:34

DevOps-Test

Set up your space

Find a conversation...

Huddles

Directories

Starred

Drag and drop important stuff here

Channels

- # all-devops-test
- # jenkins-builds
- # new-channel
- # social
- + Add channels

Direct messages

- jenkins-alerts invited you
- Sunith you
- + Invite people

Agents & apps

- Jenkins Bot
- Google Calendar
- Google Drive
- + Connect apps

jenkins-builds

Messages

jenkins-alerts 10:02 PM

joined #jenkins-builds along with 2 others.

Only visible to you

Slackbot 10:23 PM

@Jenkins Bot is already in this channel.

Only visible to you

Slackbot 10:30 PM

@Jenkins Bot is already in this channel.

Jenkins Bot APP 10:36 PM

stage_1 - #9 Started by user anonymous (Open)

stage_1 - #9 Success after 44 sec (Open)

Hello, team! Let's use this channel for...

/invite @Jenkins Bot

Conclusion

The successful deployment of Build #9 confirms that all pipeline components are fully functional and integrated.

Jenkins-Task-3

2. Create one Jenkins job using the below code and create three stages.

- **stage1:** Git clone to download the source code.
- **stage2:** Sonarqube Integration to check the quality of code.
- **stage3:** Slack Integration to send the alerts to slack.

Source Code: <https://github.com/betawins/hiring-app.git>

Objective

The primary objective of this project is to build an automated Continuous Integration (CI) pipeline that standardizes code quality audits and real-time team notifications.

Purpose

The purpose of this project is to create an automated workflow that checks code for bugs and updates the development team instantly.

Stage-1

Created a job

New Item → Name → Freestyle → Source Code Management → Git → installed git in Ec2 Jenkins master server also → Save → Build Now

stage2:

→ sudo yum -y install docker
→ systemctl start docker
→ docker pull sonarqube
→ docker images
→ docker run -itd p- 9002:9000 sonarqube
→ docker ps

To Access : [Ec2 pub_ip:9002](#)

Jenkins Settings

Jenkins → Dashboard → Manage Jenkins → Systems → SonarQube servers →

Name: jenkins_1

Server URL: <http://18.212.205.252:9002>

Server authentication token: *****

save

Server Authentication Token:

[Jenkins](#) → [Manage Jenkins](#) → [Credentials](#) → [System](#) → Global

stage3:

[Jenkins](#) → [Manage Jenkins](#) → System → Slack

Workspace: [devops-test](#)

Credential: [Select existing token which is created for Slack](#)

Default channel / member id: [#jenkins-builds](#)

Jenkins-Task-3

Custom slack app bot user: [check it](#)

Implementaion:

```
~/my
Last login: Thu Oct 1 11:28:51 2026 from 18.206.107.27
[ec2-user@ip-172-31-16-204 ~]$ sudo su
[root@ip-172-31-16-204 ec2-user]# yum install -y docker
bash: yum: command not found
[root@ip-172-31-16-204 ec2-user]# yum install -y docker
Last metadata expiration check: 7:30:49 ago on Thu Oct 1 10:38:52 2026.
Dependencies resolved.
```

Package	Architecture
Installing:	
docker	x86_64
Installing dependencies:	
container-selinux	noarch
containerd	x86_64
iptables-libs	x86_64
iptables-nft	x86_64
libcgroup	x86_64
libnetfilter_conntrack	x86_64
libnfnetlink	x86_64
libnftnl	x86_64
pigz	x86_64
runc	x86_64

Transaction Summary

Install 11 Packages

Total download size: 77 M
Installed size: 287 M
Downloading Packages:

(1/11): container-selinux-2.245.0-1.amzn2023.noarch.rpm
(2/11): iptables-libs-1.8.8-3.amzn2023.0.2.x86_64.rpm
(3/11): iptables-nft-1.8.8-3.amzn2023.0.2.x86_64.rpm
(4/11): libcgroup-3.0-1.amzn2023.0.1.x86_64.rpm
(5/11): libnetfilter_conntrack-1.0.8-2.amzn2023.0.2.x86_64.rpm
(6/11): libnfnetlink-1.0.1-19.amzn2023.0.2.x86_64.rpm
(7/11): libnftnl-1.2.2-2.amzn2023.0.2.x86_64.rpm

i-0a04c6a95573d186f (Sonarqube-server)

PublicIPs: 18.212.205.252 PrivateIPs: 172.31.16.204

```
[root@ip-172-31-16-204 ec2-user]# systemctl start docker
[root@ip-172-31-16-204 ec2-user]# docker pull sonarqube
Using default tag: latest
latest: Pulling from library/sonarqube
e3d1ed89f0d4: Pull complete
89cc57adce47: Pull complete
7130478a3071: Pull complete
4f4fb700ef54: Pull complete
e0cb59978564: Pull complete
e77c97483e9c: Pull complete
3edbd929c593: Pull complete
Digest: sha256:4905574ab8584dcac1a06c01cdc43a4ce9e369723ac4a7ab1d029c1bf71e6fe9
Status: Downloaded newer image for sonarqube:latest
docker.io/library/sonarqube:latest
[root@ip-172-31-16-204 ec2-user]# docker images
REPOSITORY TAG IMAGE ID CREATED SIZE
sonarqube latest ff13864398f3 5 days ago 1.43GB
[root@ip-172-31-16-204 ec2-user]# docker ps
CONTAINER ID IMAGE COMMAND CREATED STATUS PORTS NAMES
[root@ip-172-31-16-204 ec2-user]# docker run -itd -p 9001:9000 sonarqube
2be24ce8b263f4db159ffc3c9cff5170bebdd9a387b7c05b382d42e0f49754dd
docker: Error response from daemon: driver failed programming external connectivity on endpoint fervent_tharp (7e0f40cf59fbb6e93f1b99ff346bf455f734a11f
bind: address already in use.
[root@ip-172-31-16-204 ec2-user]# docker run -itd -p 9002:9000 sonarqube
01d60b1bc38811fa02b527459087125c711f4a7361be2cbfcb6da209430d22d5
[root@ip-172-31-16-204 ec2-user]# docker ps
CONTAINER ID IMAGE COMMAND CREATED STATUS PORTS NAMES
01d60b1bc388 sonarqube "/opt/sonarqube/dock..." 5 seconds ago Up 4 seconds 0.0.0.0:9002->9000/tcp, :::9002->9000/tcp condescending_fermi
[root@ip-172-31-16-204 ec2-user]#
```

Jenkins-Task-3

The image shows a Jenkins CI/CD pipeline execution for a project named "Sonar-container". The pipeline is running on a Jenkins server with the following configuration:

- Job Name:** Sonar-container
- Build Number:** #4
- Build Status:** SUCCESS
- Build Time:** 16.223s

The pipeline consists of the following stages and steps:

- Stage: stage_1 - #9**
 - Step: Started by user anonymous** (Open)
 - Step: Success after 44 sec** (Open)
- Stage: stage_1 - #9**
 - Step: Sonar-container - #1 Started by user anonymous** (Open)
 - Step: Sonar-container - #1 Failure after 19 sec** (Open)

The console output shows the following log messages:

```
18:29:15.374 INFO 5 source files to be analyzed for the text and secrets analysis
18:29:15.401 INFO 5/5 source files have been analyzed for the text and secrets analysis
18:29:15.405 INFO Sensor TextAndSecretsSensor [text] (done) | time=721ms
18:29:15.413 INFO ----- Run sensors on project
18:29:15.889 INFO Sensor IaC Project Sensor [iac]
18:29:15.894 INFO Sensor IaC Project Sensor [iac] (done) | time=4ms
18:29:15.895 INFO Sensor JavaProjectSensor [java]
18:29:15.899 INFO Sensor JavaProjectSensor [java] (done) | time=3ms
18:29:15.903 INFO Sensor Zero Coverage Sensor
18:29:15.906 INFO Sensor Zero Coverage Sensor (done) | time=3ms
18:29:15.906 INFO ----- Gather SCA dependencies on project
18:29:15.917 INFO Dependency analysis skipped
18:29:15.920 INFO SCM Publisher SCM provider for this project is: git
18:29:15.924 INFO SCM Publisher 4 source files to be analyzed
18:29:16.207 INFO SCM Publisher 4/4 source files have been analyzed (done) | time=278ms
18:29:16.208 INFO CPD Executor 1 file had no CPD blocks
18:29:16.209 INFO CPD Executor Calculating CPD for 0 files
18:29:16.213 INFO CPD Executor CPD calculation finished (done) | time=0ms
18:29:16.226 INFO SCM revision ID '9405b107fdffdf68e69fe953ealf9669784f9d6a'
18:29:16.617 INFO Analysis report generated in 385ms, dir size=295.1 kB
18:29:16.660 INFO Analysis report compressed in 41ms, zip size=35.2 kB
18:29:17.003 INFO Analysis report uploaded in 345ms
18:29:17.005 INFO ANALYSIS SUCCESSFUL, you can find the results at: http://18.212.205.252:9002/dashboard?id=Test
18:29:17.007 INFO Note that you will be able to access the updated dashboard once the server has processed the submitted analysis report
18:29:17.010 INFO More about the report processing at http://18.212.205.252:9002/api/ce/task?id=f0063d03-8a4a-4197-9d87-1c10dfbdf4d8
18:29:17.017 INFO Analysis total time: 7.840 s
18:29:17.027 INFO SonarScanner Engine completed successfully
18:29:17.059 INFO EXECUTION SUCCESS
18:29:17.061 INFO Total time: 16.223s
[Slack Notifications] found #3 as previous completed, non-aborted build
[Slack Notifications] will send OnBackToNormalNotification because build matches and user preferences allow it
Finished: SUCCESS
```

Jenkins-Task-3

SonarQube servers

If checked, job administrators will be able to inject a SonarQube server configuration as environment variables in the build.

☐ Environment variables

SonarQube installations

List of SonarQube installations

Name

jenkins_1

Server URL

Default is http://localhost:9000

http://18.212.205.252:9002

Server authentication token

SonarQube authentication token. Mandatory when anonymous access is disabled.

sonar-Conatiner

Advanced ▾

Save

Apply



Jenkins

/ Manage Jenkins ▾

/ Credentials

Credentials

+ Add Credentials



	jenkins_1	System - Global - jenkins_1	
	slack-token	System - Global - Slack Bot Token	
	sonar-Conatiner	System - Global - sonar-Conatiner	

Stores scoped to Jenkins

System	Domains: Global
--------	------------------

Conclusion

The successful deployment of Build confirms that all pipeline components are fully functional and integrated.